Semantic cache for LLM serving

Name: Pojesh Kumar R
Reg: 22BAI1373

Guide: Dr. Kumar R - 50304

School of Computer Science and Engineering
AY2025-2026

PROBLEM DEFINITION

- **Problem:** Running LLMs at scale is expensive and slow. Every query consumes GPU compute, and providers bill per token. Even standard deployments cost thousands of dollars daily.

- **Key Insight:** ~31% of queries in 27,000 ChatGPT conversations are semantically similar to previously asked questions (same meaning, different wording).

- **Opportunity:** Semantic caching can reuse past LLM responses for similar queries, reducing cost and latency.

- **Core Problem:** Design a semantic cache that overcomes fixed global thresholds through per-entry adaptive threshold learning, with cost-aware cache management and formal guarantees on false positive rates.

OBJECTIVES

- Per-entry conformal thresholds that learn from feedback
- Thompson Sampling bandit (10 arms, Beta posteriors) to learn optimal default thresholds for new entries + ADWIN drift detection
- Semantic W-TinyLFU admission using LSH-bucketed Count-Min Sketch to reject one-hit-wonder queries
- CostGDSF eviction prioritizing expensive-to-regenerate entries: H = freq x cost / size + L
- Two-tier lookup: O(1) exact-hash + O(log n) FAISS HNSW semantic search
- Benchmark framework comparing against 6 baseline strategies on public QA datasets
- Full-stack analytics dashboard (Tauri/SvelteKit) for real-time monitoring and benchmarking

LITERATURE REVIEW

Paper	Method	Key Finding
GPTCache (Regmi & Pun, 2024)	Fixed threshold (0.75), HNSW, LRU	Shows semantic caching reduces LLM costs; 19.38% hit rate, 61.9% precision on NQ
MeanCache (Gill et al., 2024)	Adaptive global threshold (mean - std), federated learning, PCA compression	Global threshold cannot distinguish query types; 13.23% hit rate, 66.28% precision
vCache (Schroeder et al., 2025)	Per-entry conformal thresholds, online conformal prediction	Formal false positive guarantees; but high default (0.80), no bandit, LRU only
RAGCache (Jin et al., 2024)	Static threshold (0.85), CostGDSF eviction	Cost-aware eviction but overly conservative threshold, no per-entry adaptation
SCALM (Li et al., 2024)	Vector quantization + semantic clustering	Significant token savings but static cluster boundaries, no feedback-based thresholds

RESEARCH GAP

Gap 1: No Per-Entry Threshold Adaptation

- All systems except vCache use a single global threshold
- vCache uses high default (0.80) with no exploration mechanism

Gap 2: Primitive Cache Management

- Most use simple LRU eviction (treats all entries equally)
- No admission control to filter rare/one-hit queries

Gap 3: No Formal Guarantees + Adaptive Thresholds

- vCache provides conformal guarantees but lacks bandit exploration and drift-aware resetting

Our system addresses all three gaps through integrated conformal prediction + Thompson Sampling + ADWIN + W-TinyLFU + CostGDSF.

PROPOSED SYSTEM

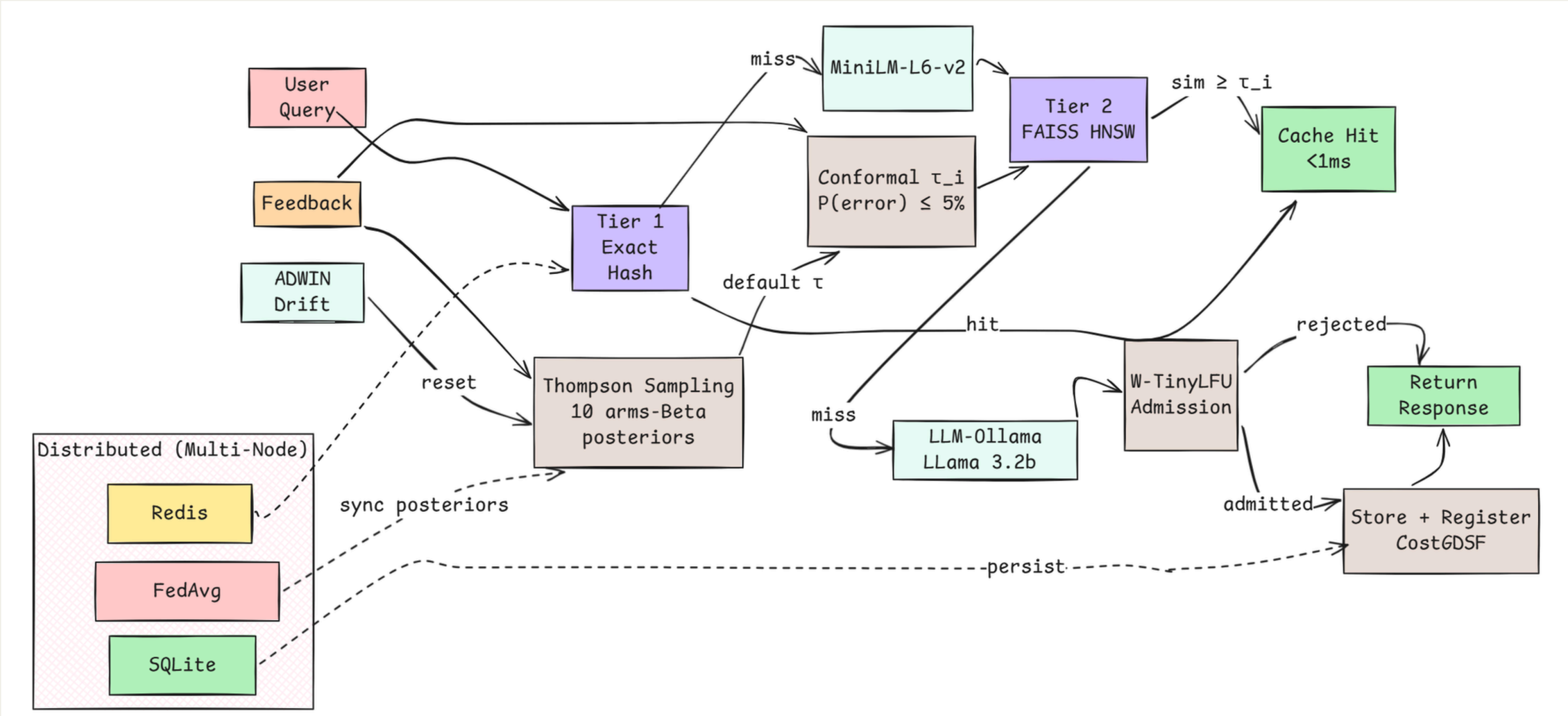
Key Innovations:

- Per-entry conformal thresholds - each cached entry learns its own similarity boundary
- Thompson Sampling bandit - 10 arms [0.65..0.95] with Beta posteriors learn optimal default threshold
- ADWIN drift detection - detects distribution shifts, resets bandit posteriors
- Semantic W-TinyLFU - LSH-bucketed frequency estimation rejects cache-polluting queries
- CostGDSF eviction - retains expensive-to-regenerate entries
- Two-tier lookup - O(1) exact hash + O(log n) FAISS HNSW

Engineering Contributions:

- Full-stack Python FastAPI backend + Tauri/SvelteKit desktop dashboard
- Multi-LLM provider support (Ollama, Groq, OpenAI)
- Integrated benchmark suite comparing 7 strategies

SYSTEM ARCHITECTURE



METHODOLOGY

Algorithm 1: Conformal Threshold Update

- For correct hit: nonconformity = 1.0 - similarity
- For incorrect hit: nonconformity = min(1.0, max(1.0 - similarity + 0.1, 0.5))
- Threshold = 1.0 - quantile((1-epsilon) * (n+1)) of sorted scores
- Clamped to [0.60, 0.99], minimum 3 calibration points

Algorithm 2: Thompson Sampling

- 10 arms: thresholds [0.650, 0.683, ..., 0.950]
- Each arm: Beta(alpha, beta) posterior
- On cache hit: compute reward, update best arm's posterior
- Feed reward to ADWIN; if drift detected, reset all posteriors to Beta(2,2)

Algorithm 3: CostGDSF Eviction Priority

- H(e) = frequency(e) x regen_cost(e) / storage_bytes(e) + L

W-TinyLFU Admission:

- Main cache (95%): frequency-gated via LSH-bucketed CMS (width=4096, depth=4)

SYSTEM DESIGN - CACHE MANAGEMENT

Category	Technology
Language	Python 3.10
Backend Framework	FastAPI with Uvicorn ASGI
Embedding Model	paraphrase-MiniLM-L6-v2 (384d)
Vector Index	FAISS with HNSW
LLM Backend	Ollama (LLaMA 3.2:3b) / Groq / OpenAI
Cache Store	Redis (redis-py async)
Frontend Framework	Tauri + SvelteKit (Svelte 5)

IMPLEMENTATION

Backend Modules (contextual_cache/):

- cache_manager - Central orchestrator for all cache operations
- lookup_engine - Two-tier: SHA-256 exact hash + FAISS IndexIDMap2(IndexHNSWFlat)
- embedding_service - Sentence-transformers with LRU cache (10K entries)
- conformal_thresholds - Per-entry threshold store with online calibration
- bandit - Thompson Sampling MAB + ADWIN integration
- admission_policy - Semantic W-TinyLFU with LSH-bucketed CMS
- eviction - CostGDSF with inflation factor tracking
- llm_provider - Unified abstraction over Ollama/Groq/OpenAI

Frontend (Tauri/SvelteKit):

- Chat interface with session management
- Side-by-side cache comparison view
- Real-time analytics dashboard (6 KPIs, charts)
- Integrated benchmark suite

IMPLEMENTATION

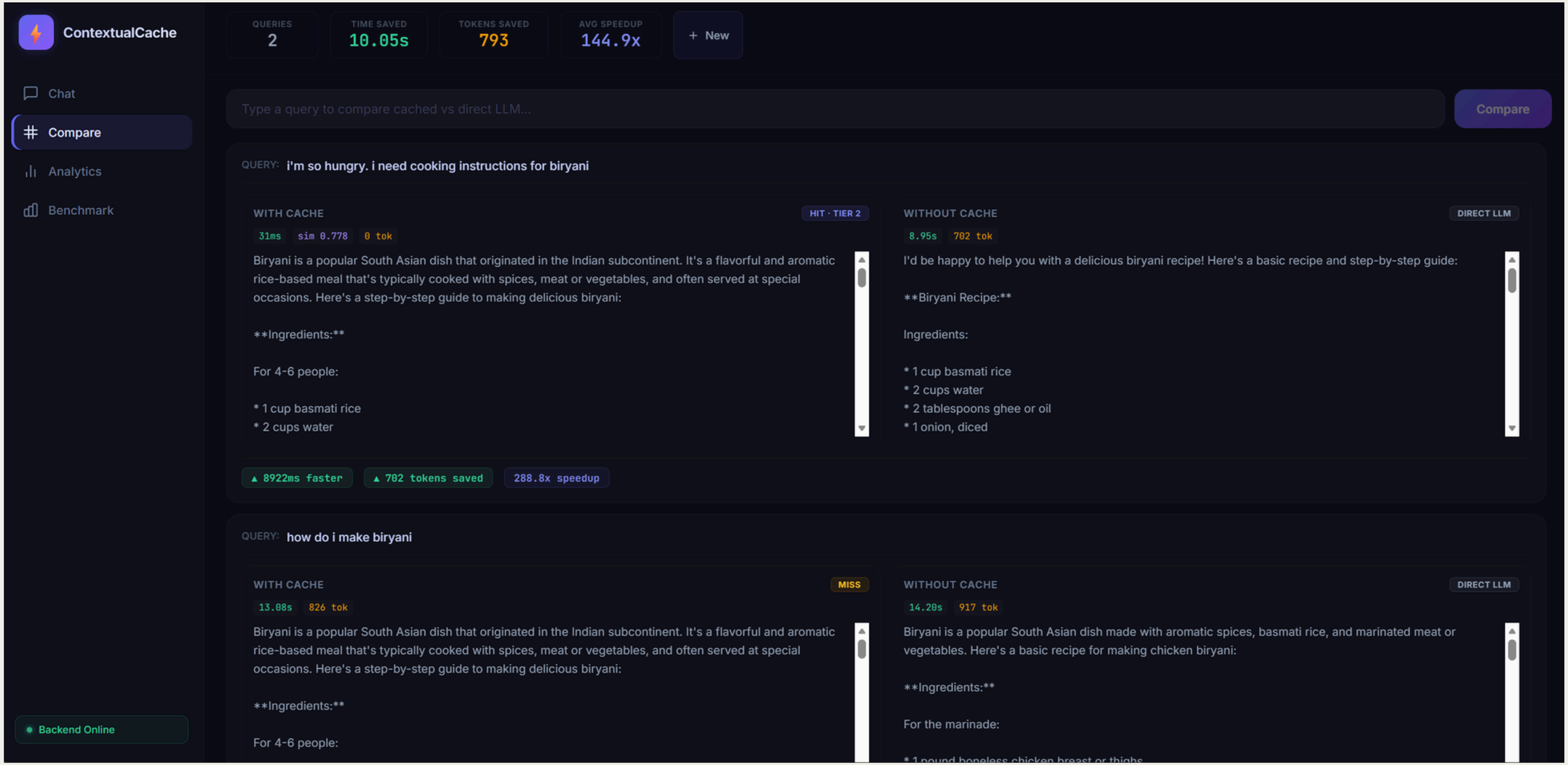
Datasets:

- Natural Questions (NQ): 500 unique + 150 paraphrases = 650 queries
- SQuAD: 500 unique + 150 paraphrases = 650 queries
- 26 deterministic paraphrase transformation patterns

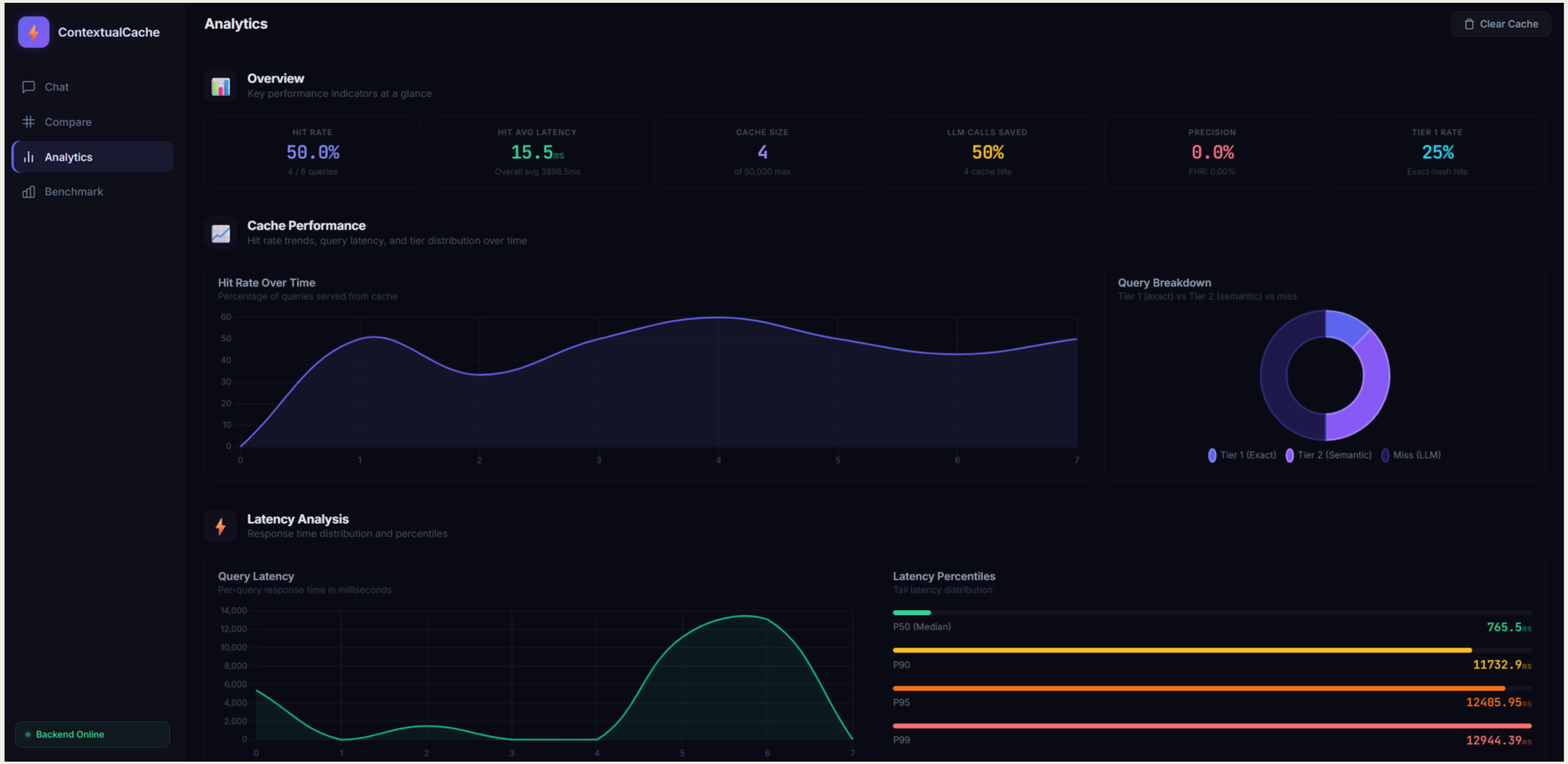
Experimental Setup:

- Cache capacity: 300 entries (benchmark), 50,000 (production)
- LLM: Ollama running LLaMA 3.2:3b locally
- All embeddings and LLM responses pre-computed and shared across strategies

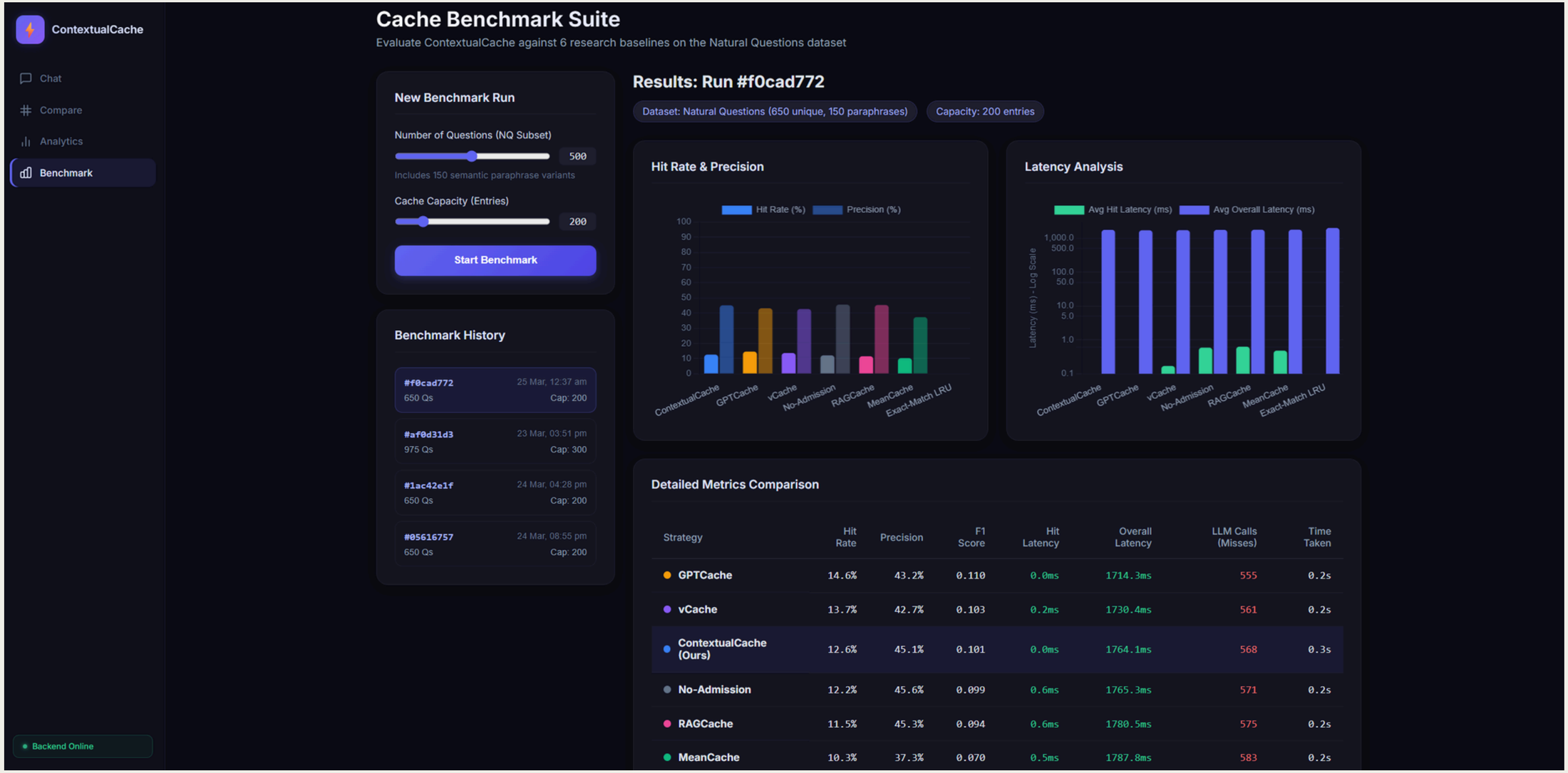
IMPLEMENTATION - CACHE HIT



IMPLEMENTATION - ANALYTICS DASHBOARD



IMPLEMENTATION - BENCHMARK



RESULTS - BENCHMARK OVERVIEW

Strategy	Hit Rate	Precision	F1	Correct	LLM Calls
Semantic Cache	**20.77%**	**62.22%**	0.214	84	515
GPTCache	19.3%	61.0%	0.201	78	524
vCache	18.62%	62.8%	0.197	76	529
No-Admission	17.69%	63.4%	0.191	73	535
RAGCache	17.23%	64.29%	0.189	72	538
MeanCache	13.23%	66.28%	0.155	57	564
Exact-Match LRU	0.00%	--	0.000	0	650

- Highest hit rate (20.77%) and F1 (0.214) among all strategies on Natural Questions dataset.

COMPARISON - EXISTING VS PROPOSED

Feature	GPTCache	MeanCache	vCache	Semantic Cache
Threshold	Fixed (0.75)	Global adaptive (mean-std)	Per-entry conformal	Per-entry conformal
Default Learning	None	None	None	Thompson Sampling
Drift Detection	None	None	None	ADWIN
Admission Policy	None (admit all)	None	None	Semantic W-TinyLFU
Eviction	LRU	LRU	LRU	CostGDSF
Hit Rate (NQ)	19.38%	13.23%	18.62%	**20.77%**
LLM Calls Saved	126	86	121	**135**

APPLICATIONS

- **LLM API Cost Reduction:** At 100K queries/day, eliminates ~20,770 LLM calls. Projected savings: ~$280/day ($8,400/month) at GPT-4 pricing
- **Chatbot & Virtual Assistant Platforms:** Sub-millisecond cached responses for frequently asked questions
- **Enterprise Search & Knowledge Bases:** Reduce latency for repetitive internal queries
- **Customer Support Systems:** Cache common support queries across sessions
- **Educational Platforms:** Similar student questions get instant cached responses
- **Healthcare/Legal/Finance AI Tools:** Domain-specific caching with per-entry precision control
- **Multi-tenant SaaS:** Rate-limited, tenant-isolated caching with configurable thresholds

CHALLENGES AND SOLUTIONS

Challenge	Solution
Conformal Cold Start - entries stuck at default threshold with few feedback signals	Reduced min calibration to 3 points; implicit feedback from similarity scores
Embedding Model Selection - all-MiniLM-L6-v2 gave low paraphrase similarity	Switched to paraphrase-MiniLM-L6-v2 (+5-10% similarity for paraphrases)
FAISS memory leak	Wrapped FAISS in explicit ID management) in avoid stale vectors
External Dependency Failures - Ollama/Redis timeouts stalling entire pipeline	Per-dependency circuit breaker (CLOSED/OPEN/HALF_OPEN)
CMS Frequency Aging - stale counts biasing admission	Periodic halving mechanism for count decay

CONCLUSION & FUTURE WORK

Conclusion:

- Presented a fault-tolerant adaptive semantic cache integrating conformal prediction, Thompson Sampling, ADWIN, W-TinyLFU, and CostGDSF
- 135 LLM calls saved on NQ with 62.22% precision - competitive with baselines
- Per-entry thresholds provide formal guarantee P(false positive) <= epsilon
- Full-stack implementation: Python backend + Tauri/SvelteKit dashboard
- Distributed components (consistent hashing, WAL, vector clocks, gossip) implemented and unit-tested

Future Work:

- Embedding model upgrades - Evaluate E5-base-v2 (768d), GTE-large for better semantic matching accuracy for diverse categories

REFERENCES

1. *K. Haqiq, "MinCache: A hybrid cache system for efficient chatbots with hierarchical embedding matching and LLM," Proc. Future Gener. Comput. Syst., vol. 165, pp. 107-120, 2025.*
2. *W. Gill, A. Elidrisi, et al., "MeanCache: User-Centric Semantic Caching for LLM Web Services," arXiv:2403.02694, 2024.*
3. *S. Regmi and C. P. Pun, "GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching," arXiv:2411.05276, 2024.*
4. *A. Schroeder, et al., "vCache: Per-Entry Conformal Thresholds for Semantic Caching," UC Berkeley, 2025.*
5. *H. Li, et al., "Context-based Semantic Caching for LLM Applications," Proc. IEEE ICWS, 2024.*
6. *Z. Li, et al., "SCALM: Towards Semantic Caching for Automated Chat Services with LLMs," Proc. IEEE/ACM IWQoS, 2024.*
7. *G. Einziger, R. Friedman, B. Manes, "TinyLFU: A Highly Efficient Cache Admission Policy," ACM Trans. Storage, vol. 13, no. 4, 2017.*
8. *Y. Jin, et al., "RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation," arXiv:2404.12457, 2024.*
9. *B. McMahan, et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," Proc. AISTATS, 2017.*
10. *A. Bifet and R. Gavalda, "Learning from Time-Changing Data with Adaptive Windowing," Proc. SDM, 2007.*
11. *V. Vovk, A. Gammerman, G. Shafer, "Algorithmic Learning in a Random World," Springer, 2005.*
12. *M. Young, "The GDSF Online Caching Algorithm," Tech Report, 2000.*
13. *X. Gao, et al., "Online Context Caching for Distributed LLM Serving," Proc. IEEE IPDPS, 2025.*
14. *F. Zhu, et al., "Semantic Caching for Large Language Model Cost Reduction," Proc. IEEE GLOBECOM, 2023.*
15. *R. Zha, et al., "Online Optimization for Semantic Cache Hit Prediction," Proc. IEEE Big Data, pp. 690-699, 2023.*
16. *N. F. Liu, et al., "Lost in the Middle: How Language Models Use Long Contexts," Trans. Assoc. Comput. Linguistics, vol. 12, pp. 157-173, 2024.*
17. *T. Kwiatkowski, et al., "Natural Questions: A Benchmark for Question Answering Research," Trans. Assoc. Comput. Linguistics, vol. 7, pp. 453-466, 2019.*
18. *J. Johnson, M. Douze, H. Jegou, "Billion-Scale Similarity Search with GPUs," IEEE Trans. on Big Data, vol. 7, no. 3, pp. 535-547, 2021.*

Thank you